

PRATICAL 3 &4

PART 1 — Multiple children using wait()

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;

    printf("Parent PID: %d\n", getpid());

    for (i = 0; i < 3; i++)
    {
        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0)
        {
            printf("Child %d created: PID = %d, PPID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d terminating: PID = %d\n",
                i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("\nParent is waiting for children using wait().\n");

    for (i = 0; i < 3; i++)
    {
        int status;
        pid_t child_pid;

        child_pid = wait(&status);

        printf("Parent collected child PID = %d\n",
            child_pid);

        if (WIFEXITED(status))
        {
            printf("Exit status = %d\n",
                WEXITSTATUS(status));
        }
    }

    printf("All children have been collected.\n");
    printf("Parent terminating.\n");

    return 0;
}
```

```

hemasai@Hemasai:~/OSSP_Assignment$ mkdir -p Practical_3_4
hemasai@Hemasai:~/OSSP_Assignment$ cd Practical_3_4
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ nano multiple_wait.c
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ gcc multiple_wait.c -o multiple_wait
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ ./multiple_wait
Parent PID: 1469
Child 1 created: PID = 1470, PPID = 1469

Parent is waiting for children using wait().
Child 3 created: PID = 1472, PPID = 1469
Child 2 created: PID = 1471, PPID = 1469
Child 1 terminating: PID = 1470
Parent collected child PID = 1470
Exit status = 10
Child 2 terminating: PID = 1471
Parent collected child PID = 1471
Exit status = 11
Child 3 terminating: PID = 1472
Parent collected child PID = 1472
Exit status = 12
All children have been collected.
Parent terminating.
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ |

```

PART 2 — Multiple children using waitpid()

```

GNU nano 3.7.1 multiple_waitpid.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t children[3];
    int i;

    printf("Parent PID: %d\n", getpid());

    for (i = 0; i < 3; i++)
    {
        children[i] = fork();
        if (children[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (children[i] == 0)
        {
            printf("Child %d created: PID = %d, PPID = %d\n",
                i + 1, getpid(), getppid());

            sleep(3 - i);

            printf("Child %d terminating: PID = %d\n",
                i + 1, getpid());

            exit(20 + i);
        }
    }

    printf("\nParent is waiting for specific children using waitpid().\n");
    for (i = 0; i < 3; i++)
    {
        int status;
        waitpid(children[i], &status, 0);
        printf("Parent collected specific child PID = %d\n",
            children[i]);

        if (WIFEXITED(status))
        {
            printf("Exit status = %d\n",
                WEXITSTATUS(status));
        }
    }

    printf("All children have been collected.\n");
    printf("Parent terminating.\n");

    return 0;
}

```

```

hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ nano multiple_waitpid.c
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ gcc multiple_waitpid.c -o multiple_waitpid
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ ./multiple_waitpid
Parent PID: 1510
Child 1 created: PID = 1511, PPID = 1510
Child 2 created: PID = 1512, PPID = 1510

Parent is waiting for specific children using waitpid().
Child 3 created: PID = 1513, PPID = 1510
Child 3 terminating: PID = 1513
Child 2 terminating: PID = 1512
Child 1 terminating: PID = 1511
Parent collected specific child PID = 1511
Exit status = 20
Parent collected specific child PID = 1512
Exit status = 21
Parent collected specific child PID = 1513
Exit status = 22
All children have been collected.
Parent terminating.
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$

```

PART 3 — Zombie Process

```
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID: %d\n", getppid());

        printf("Child is terminating.\n");

        exit(0);
    }
    else
    {
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("Parent is NOT calling wait().\n");
        printf("Child will become a zombie.\n");

        sleep(20);

        printf("Parent terminating.\n");
    }

    return 0;
}
```

```
Parent terminating.
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ nano zombie.c
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ gcc zombie.c -o zombie
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ ./zombie &
[1] 1549

--- PARENT PROCESS ---
Parent PID : 1549
Child PID : 1552
Parent is NOT calling wait().
Child will become a zombie.

--- CHILD PROCESS ---
Child PID : 1552
Child PPID: 1549
Child is terminating.
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ |
```

```

Child PPID: 1549
Child is terminating.
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ Parent terminating.
^C
[1]+  Done                  ./zombie
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ ps -o pid,ppid,state,stat,cmd -p <CHILD_PID>
-bash: syntax error near unexpected token 'newline'
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ ps -o pid,ppid,state,stat,cmd -p <CHILD_PID>
-bash: syntax error near unexpected token 'newline'
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ ./zombie &
[1] 1575

--- PARENT PROCESS ---
Parent PID : 1575
Child PID  : 1577
Parent is NOT calling wait().
Child will become a zombie.

--- CHILD PROCESS ---
Child PID : 1577
Child PPID: 1575
Child is terminating.
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ Parent terminating.
^C
[1]+  Done                  ./zombie
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ ps -o pid,ppid,state,stat,cmd -p 1577
  PID  PPID  S  STAT  CMD
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ |

```

eliminate the zombie using wait().

```

hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ nano zombie_fixed.c
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ gcc zombie_fixed.c -o zombie_fixed
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ ./zombie_fixed

--- PARENT PROCESS ---
Parent PID : 1641
Child PID  : 1642
Parent is waiting for child using wait().

--- CHILD PROCESS ---
Child PID : 1642
Child PPID: 1641
Child is terminating.
Child has been collected.
Zombie process eliminated.
Parent terminating.
hemasai@Hemasai:~/OSSP_Assignment/Practical_3_4$ |

```

```

#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID: %d\n", getppid());
        printf("Child is terminating.\n");

        exit(0);
    }
    else
    {
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("Parent is waiting for child using wait().\n");

        wait(NULL);

        printf("Child has been collected.\n");
        printf("Zombie process eliminated.\n");
        printf("Parent terminating.\n");
    }

    return 0;
}

```